

Кодирование

Объясняется, как Protocol Buffers кодирует данные для сохранения в файлах или передачи по сети.

В этом документе описан буфер протокола в *проводном формате*, который определяет, как именно ваше сообщение передается по сети и сколько места оно занимает на диске. Скорее всего, вам не нужно разбираться в этом, чтобы использовать буферы протокола в своем приложении, но эта информация будет полезна для оптимизации.

Если вы уже знакомы с этими понятиями, но хотите свериться с ними, перейдите к разделу [Краткая справочная информация](#).

[Protoscope](#) — это очень простой язык для описания фрагментов низкоуровневого формата передачи данных, который мы будем использовать в качестве визуального ориентира для кодирования различных сообщений. Синтаксис Protoscope состоит из последовательности *токенов*, каждый из которых кодируется до определенной последовательности байтов.

Например, обратные кавычки обозначают необработанный шестнадцатеричный литерал, например ``70726f746662756660a``. Он кодируется в те же байты, которые указаны в шестнадцатеричном литерале. Кавычки обозначают строки в кодировке UTF-8, например `"Hello, Protobuf!"`. Этот литерал является синонимом ``48656c6c662c2050726f7466627566621`` (который, если присмотреться, состоит из байтов ASCII). Мы расскажем больше о языке Protoscope по мере обсуждения аспектов формата передачи данных.

Инструмент Protoscope также может выводить закодированные буферы протокола в виде текста. Примеры см.

<https://github.com/protocolbuffers/protoscope/tree/main/testdata>.

Все примеры в этом разделе предполагают, что вы используете версию 2023 или более позднюю.

Простое сообщение

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа



```
message Test1 {
  int32 a = 1;
}
```

В приложении вы создаете сообщение `Test1` и устанавливаете `a` значение 150. Затем вы сериализуете сообщение в выходной поток. Если бы у вас была возможность изучить закодированное сообщение, вы бы увидели три байта:



```
08 96 01
```

Пока все так просто — маленькие числа, но что они означают? Если вы воспользуетесь инструментом Protoscope для вывода этих байтов, то получите что-то вроде `1: 150`. Откуда он знает, что это содержимое сообщения?

Варианты с основанием 128

Целые числа переменной ширины, или *варианты*, лежат в основе формата wire. Они позволяют кодировать 64-битные целые числа без знака, используя от одного до десяти байт, при этом для небольших значений требуется меньше байт.

Каждый байт в `varint` имеет *продолжающий бит*, который указывает, является ли следующий байт частью `varint`. Это *старший бит* (MSB) байта (иногда его также называют *знаковым битом*). Младшие 7 бит — это полезная нагрузка; результирующее целое число формируется путем объединения 7-битных полезных нагрузок составляющих его байтов.

Например, вот число 1, закодированное как ``01`` — это один байт, поэтому старший бит не установлен:



```
0000 0001
^ msb
```

А вот 150, закодированное как ``9601`` — это немного сложнее:



```
10010110 00000001
```

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

Как понять, что это 150? Сначала нужно отбросить старший бит каждого байта, поскольку он нужен только для того, чтобы понять, достигли ли мы конца числа (как видно, он установлен в первом байте, поскольку в varint больше одного байта). Эти 7-битные данные записаны в порядке следования байтов от младшего к старшему. Преобразуем их в порядок следования байтов от старшего к младшему, объединяем и интерпретируем как 64-битное целое число без знака:

```
10010110 00000001 // Original inputs.
0010110 0000001 // Drop continuation bits.
0000001 0010110 // Convert to big-endian.
00000010010110 // Concatenate.
128 + 16 + 4 + 2 = 150 // Interpret as an unsigned 64-bit integer.
```

Поскольку varints играют важнейшую роль в буферах протокола, в синтаксисе protoscore мы указываем их как обычные целые числа. 150 — это то же самое, что `9601`.

Структура сообщения

Сообщение в буфере протокола представляет собой набор пар «ключ-значение». В двоичной версии сообщения в качестве ключа используется только номер поля — имя и объявленный тип каждого поля можно определить только на этапе декодирования, обратившись к определению типа сообщения (то есть к файлу .proto). У Protoscore нет доступа к этой информации, поэтому он может предоставить только номера полей.

При кодировании сообщения каждая пара «ключ-значение» преобразуется в запись, состоящую из номера поля, типа передачи и полезной нагрузки. Тип передачи сообщает парсеру, какого размера будет полезная нагрузка. Это позволяет старым парсерам пропускать новые поля, которые они не понимают. Такой тип схемы иногда называют [Tag-Length-Value](#), или TLV.

Существует шесть типов проводов: VARINT, I64, LEN, SGROUP, EGROUP, и I32

ID	Имя	Используется Для
0	ВАРИНТ	int32, int64, uint32, uint64, sint32, sint64, bool, enum
1	I64	fixed64, sfixed64, double
2	ЛЕН	строка, байты, встроенные сообщения, упакованные

ID	Имя	Используется Для
3	СГРУППИРОВКА	групповой старт (устарел)
4	ЭГРУППА	конец группы (устарел)
5	I32	fixed32, sfixed32, float

«Тег» записи кодируется как `varint`, образованный из номера поля и типа провода по формуле $(\text{field_number} \ll 3) \mid \text{wire_type}$. Другими словами, после декодирования `varint`, представляющего поле, младшие 3 бита сообщают нам тип провода, а оставшаяся часть целого числа — номер поля.

Теперь давайте еще раз рассмотрим наш простой пример. Теперь вы знаете, что первое число в потоке всегда является ключом `varint`, и здесь это ``08``, или (без старшего бита):

000 1000



Вы берёте последние три бита, чтобы получить тип провода (0), а затем сдвигаете вправо на три бита, чтобы получить номер поля (1). Protoscope представляет тег в виде целого числа, за которым следует двоеточие и тип провода, поэтому мы можем записать приведенные выше байты как `1:VARINT`.

Поскольку тип провода равен 0 или `VARINT`, мы знаем, что для получения полезной нагрузки нам нужно декодировать `varint`. Как мы видели выше, байты ``9601`` декодируются в `varint` как 150, что дает нам нашу запись. В Protoscope мы можем записать ее как `1:VARINT 150`.

Protoscope может определить тип тега, если после `:` есть пробел. Для этого он просматривает следующий токен и пытается угадать, что вы имели в виду (правила подробно описаны в [языковом файле Protoscope's language.txt](#)). Например, в `1: 150` сразу после нетипизированного тега следует `varint`, поэтому Protoscope определяет его тип как `VARINT`. Если бы вы написали `2: {}`, он увидел бы `{` и предположил бы, что это `LEN`; если бы вы написали `3: 5i32`, он предположил бы `I32`, и так далее.

Другие целочисленные типы

Булевы значения и перечисления

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

Целочисленные значения со знаком

Как вы видели в предыдущем разделе, все типы буфера протокола, связанные с типом провода 0, кодируются как `varints`. Однако `varints` — это беззнаковые типы, поэтому разные типы со знаком, `sint32` и `sint64` против `int32` или `int64`, кодируют отрицательные числа по-разному.

Типы `intN` кодируют отрицательные числа в дополнительном коде, то есть в 64-битных целых числах без знака старший бит установлен. В результате это означает, что должны быть использованы *все десять байт*. Например, `-2` преобразуется `protoscope` в

11111110 11111111 11111111 11111111 11111111
11111111 11111111 11111111 11111111 00000001

Это *дополнение до двух* числа 2, которое в арифметике без знака определяется как $\sim 0 - 2 + 1$, где ~ 0 — 64-битное целое число, состоящее из одних единиц. Полезно понять, почему получается так много единиц.

`sintN` Для кодирования отрицательных целых чисел используется кодировка «зигзаг» вместо дополнения до двух. Положительные целые числа p кодируются как $2 * p$ (четные числа), а отрицательные целые числа n — как $2 * |n| - 1$ (нечетные числа). Таким образом, кодировка «зигзагообразна» в отношении положительных и отрицательных чисел. Например:

Подписанный Оригинал	Кодируется как
0	0
-1	1
1	2
-2	3
...	...
0x7fffffff	0xffffffffe
-0x80000000	0xfffffffff

```
(n << 1) ^ (n >> 31)
```

для `sint32` `s`, или

```
(n << 1) ^ (n >> 63)
```

для 64-битной версии.

При разборе `sint32` или `sint64` его значение декодируется обратно в исходную версию со знаком.

В Protoscope добавление суффикса `z` к целому числу приводит к его кодированию в формате ZigZag. Например, `-500z` — это то же самое, что и `varint 999`.

Числа без varint

Числа без `varint` устроены просто. `double` и `fixed64` имеют тип `wire I64`, который указывает парсеру на необходимость ожидать фиксированный восьмибайтовый фрагмент данных. Значения `double` закодированы в формате двойной точности IEEE 754. Мы можем указать запись `double` с помощью 5: `25.4`, а запись `fixed64` — с помощью 6: `200i64`.

Аналогичным образом `float` и `fixed32` имеют тип `wire I32`, который указывает на необходимость ожидать четыре байта. `float` Значения кодируются в формате одинарной точности IEEE 754. Синтаксис для них состоит в добавлении суффикса `i32`. `25.4i32` будет выдавать четыре байта, как и `200i32`. Типы тегов определяются как `I32`.

Length-Delimited Records

Length prefixes are another major concept in the wire format. The `LEN` wire type has a dynamic length, specified by a varint immediately after the tag, which is followed by the payload as usual.

Consider this message schema:

```
message Test2 {  
  string b = 2;  
}
```

трафика. Узнать больше

A record for the field `b` is a string, and strings are `LEN`-encoded. If we set `b` to `"testing"`, we encoded as a `LEN` record with field number 2 containing the ASCII string `"testing"`. The result is ``12 07 [74 65 73 74 69 6e 67]``. Breaking up the bytes,

12 07 [74 65 73 74 69 6e 67]

we see that the tag, ``12``, is `00010 010`, or `2:LEN`. The byte that follows is the int32 varint `7`, and the next seven bytes are the UTF-8 encoding of `"testing"`. The int32 varint means that the max length of a string is 2GB.

In Protoscope, this is written as `2:LEN 7 "testing"`. However, it can be inconvenient to repeat the length of the string (which, in Protoscope text, is already quote-delimited). Wrapping Protoscope content in braces will generate a length prefix for it: `{"testing"}` is a shorthand for `7 "testing"`. `{}` is always inferred by fields to be a `LEN` record, so we can write this record simply as `2: {"testing"}`.

bytes fields are encoded in the same way.

Submessages

Submessage fields also use the `LEN` wire type. Here's a message definition with an embedded message of our original example message, `Test1`:

```
message Test3 {  
  Test1 c = 3;  
}
```

If `Test1`'s a field (i.e., `Test3`'s `c`.a field) is set to 150, we get ``1a 03 [08 96 01]``. Breaking it up:

1a 03 [08 96 01]

The last three bytes (in `[]`) are exactly the same ones from our [very first example](#). These bytes are preceded by a `LEN`-typed tag, and a length of 3, exactly the same way as strings are encoded.

In Protoscope, submessages are quite succinct. ```1a03089601``` can be written as `3: {1: 150}`.

Missing Elements

Missing fields are easy to encode: we just leave out the record if it's not present. This means that “huge” protos with only a few fields set are quite sparse.

Repeated Elements

Starting in Edition 2023, repeated fields of a primitive type (any [scalar type](#) that is not `string` or `bytes`) are [“packed”](#) by default.

Packed repeated fields, instead of being encoded as one record per entry, are encoded as a single `LEN` record that contains each element concatenated. To decode, elements are decoded from the `LEN` record one by one until the payload is exhausted. The start of the next element is determined by the length of the previous, which itself depends on the type of the field. Thus, if we have:

```
message Test4 {
  string d = 4;
  repeated int32 e = 5;
}
```

and we construct a `Test4` message with `d` set to `"hello"`, and `e` set to `1`, `2`, and `3`, this *could* be encoded as ``220568656c6c6f2a03010203``, or written out as Protoscope,

```
4: {"hello"}
5: {1 2 3}
```

However, if the repeated field is set to expanded (overriding the default packed state) or is not packable (strings and messages) then an entry for each individual value is encoded. Also, records for `e` do not need to appear consecutively, and can be interleaved with other fields.

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

only the order of records for the same field with respect to each other is preserved. Thus, this could look like the following:

```
5: 1
5: 2
4: {"hello"}
5: 3
```

Only repeated fields of primitive numeric types can be declared “packed”. These are types that would normally use the `VARINT`, `I32`, or `I64` wire types.

Note that although there’s usually no reason to encode more than one key-value pair for a packed repeated field, parsers must be prepared to accept multiple key-value pairs. In this case, the payloads should be concatenated. Each pair must contain a whole number of elements. The following is a valid encoding of the same message above that parsers must accept:

```
5: {1 2}
4: {"hello"}
5: {3}
```

Protocol buffer parsers must be able to parse repeated fields that were compiled as `packed` as if they were not packed, and vice versa. This permits adding `[packed=true]` to existing fields in a forward- and backward-compatible way.

Oneofs

[Oneof fields](#) are encoded the same as if the fields were not in a `oneof`. The rules that apply to `oneofs` are independent of how they are represented on the wire.

Last One Wins

Normally, an encoded message would never have more than one instance of a non-repeated field. However, parsers are expected to handle the case in which they do. For numeric types and strings, if the same field appears multiple times, the parser accepts the *last* value it sees. For embedded message fields, the parser merges multiple instances of the same field, as if with the `Message::MergeFrom` method – that is, all singular scalar fields in the latter instance replace those in the former, singular embedded messages are merged, and

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

of two encoded messages produces exactly the same result as if you had parsed the two messages separately and merged the resulting objects. That is, this:

```
MyMessage message;  
message.ParseFromString(str1 + str2);
```

is equivalent to this:

```
MyMessage message, message2;  
message.ParseFromString(str1);  
message2.ParseFromString(str2);  
message.MergeFrom(message2);
```

This property is occasionally useful, as it allows you to merge two messages (by concatenation) even if you do not know their types.

Maps

Map fields are just a shorthand for a special kind of repeated field. If we have

```
message Test6 {  
  map<string, int32> g = 7;  
}
```

this is actually the same as

```
message Test6 {  
  message g_Entry {  
    string key = 1;  
    int32 value = 2;  
  }  
  repeated g_Entry g = 7;  
}
```

Thus, maps are encoded almost exactly like a `repeated` message field: as a sequence of

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял
предоставления и повышения качества своих услуг, а также для анализа

Groups

трафика. Узнать больше

Groups are a deprecated feature that should not be used, but they remain in the wire format, and deserve a passing mention.

A group is a bit like a submessage, but it is delimited by special tags rather than by a `LEN` prefix. Each group in a message has a field number, which is used on these special tags.

A group with field number `8` begins with an `8:SGROUP` tag. `SGROUP` records have empty payloads, so all this does is denote the start of the group. Once all the fields in the group are listed, a corresponding `8:EGROUP` tag denotes its end. `EGROUP` records also have no payload, so `8:EGROUP` is the entire record. Group field numbers need to match up. If we encounter `7:EGROUP` where we expect `8:EGROUP`, the message is mal-formed.

Protoscope provides a convenient syntax for writing groups. Instead of writing

```
8:SGROUP
  1: 2
  3: {"foo"}
8:EGROUP
```

Protoscope allows

```
8: !{
  1: 2
  3: {"foo"}
}
```

This will generate the appropriate start and end group markers. The `!{ }` syntax can only occur immediately after an un-typed tag expression, like `8: .`

Field Order

Field numbers may be declared in any order in a `.proto` file. The order chosen has no effect on how the messages are serialized.

When a message is serialized, there is no guaranteed order for how its known or [unknown fields](#) will be written. Serialization order is an implementation detail, and the details of any particular implementation may change in the future. Therefore, we should not

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

- Do not assume the byte output of a serialized message is stable. This is especially true for messages with transitive bytes fields representing other serialized protocol buffer messages.
- By default, repeated invocations of serialization methods on the same protocol buffer message instance may not produce the same byte output. That is, the default serialization is not deterministic.
 - Deterministic serialization only guarantees the same byte output for a particular binary. The byte output may change across different versions of the binary.
- The following checks may fail for a protocol buffer message instance `foo` :
 - `foo.SerializeAsString() == foo.SerializeAsString()`
 - `Hash(foo.SerializeAsString()) == Hash(foo.SerializeAsString())`
 - `CRC(foo.SerializeAsString()) == CRC(foo.SerializeAsString())`
 - `FingerPrint(foo.SerializeAsString()) == FingerPrint(foo.SerializeAsString())`
- Here are a few example scenarios where logically equivalent protocol buffer messages `foo` and `bar` may serialize to different byte outputs:
 - `bar` is serialized by an old server that treats some fields as unknown.
 - `bar` is serialized by a server that is implemented in a different programming language and serializes fields in different order.
 - `bar` has a field that serializes in a non-deterministic manner.
 - `bar` has a field that stores a serialized byte output of a protocol buffer message which is serialized differently.
 - `bar` is serialized by a new server that serializes fields in a different order due to an implementation change.
 - `foo` and `bar` are concatenations of the same individual messages in a different order.

Encoded Proto Size Limitations

Protos must be smaller than 2 GiB when serialized. Many proto implementations will refuse to serialize or parse messages that exceed this limit.

Condensed Reference Card

The following provides the most prominent parts of the wire format in an easy-to-reference format.

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

```

message      := (tag value)* трафика. Узнать больше

tag          := (field << 3) bit-or wire_type;
               encoded as uint32 varint
value        := varint      for wire_type == VARINT,
               i32         for wire_type == I32,
               i64         for wire_type == I64,
               len-prefix  for wire_type == LEN,
               <empty>     for wire_type == SGROUP or EGROUP

varint       := int32 | int64 | uint32 | uint64 | bool | enum | sint32 | sint64;
               encoded as varints (sintN are ZigZag-encoded first)
i32          := sfixed32 | fixed32 | float;
               encoded as 4-byte little-endian (float is IEEE 754
               single-precision); memcpy of the equivalent C types (u?int32_t,
               float)
i64          := sfixed64 | fixed64 | double;
               encoded as 8-byte little-endian (double is IEEE 754
               double-precision); memcpy of the equivalent C types (u?int64_t,
               double)

len-prefix   := size (message | string | bytes | packed);
               size encoded as int32 varint
string       := valid UTF-8 string (e.g. ASCII);
               max 2GB of bytes
bytes        := any sequence of 8-bit bytes;
               max 2GB of bytes
packed       := varint* | i32* | i64*,
               consecutive values of the type specified in `.proto`

```

See also the [Protoscope Language Reference](#).

Key

message := (tag value)*

A message is encoded as a sequence of zero or more pairs of tags and values.

tag := (field << 3) bit-or wire_type

A tag is a combination of a **wire_type**, stored in the least significant three bits, and the field number that is defined in the **.proto** file.

value := varint for wire_type == VARINT, ...

A value is stored differently depending on the **wire_type** specified in the tag.

varint := int32 | int64 | uint32 | uint64 | bool | enum | sint32 | sint64

You can use varint to store any of the listed data types

Protobuf.разработчик использует файлы cookie от Google для Ладно, понял предоставления и повышения качества своих услуг, а также для анализа

You can use `fixed32` to store any of the listed data types.

`i64 := sfixed64 | fixed64 | double`

You can use `fixed64` to store any of the listed data types.

`len-prefix := size (message | string | bytes | packed)`

A length-prefixed value is stored as a length (encoded as a varint), and then one of the listed data types.

`string := valid UTF-8 string (e.g. ASCII)`

As described, a string must use UTF-8 character encoding. A string cannot exceed 2GB.

`bytes := any sequence of 8-bit bytes`

As described, bytes can store custom data types, up to 2GB in size.

`packed := varint* | i32* | i64*`

Use the `packed` data type when you are storing consecutive values of the type described in the protocol definition. The tag is dropped for values after the first, which amortizes the costs of tags to one per field, rather than per element.